

ARK — Engineering Execution Plan (EEP)

Status: **FINAL** — the operating manual for implementation. Precedes: implementation only. No further planning documents will be created.

Sources of truth (LOCKED, do not redesign):

1. `docs/engineering-companion-phase1.md` — Engineering Companion Foundation
2. `docs/developer-companion-experience.md` — Developer Companion Experience (DCX)
3. `docs/ark-information-architecture.md` — Information Architecture (IA)
4. `docs/ecis.md` — Engineering Companion Implementation Specification (ECIS)

Binding constraints (from the locked documents, restated here for enforcement):

- No backend redesign. No database redesign. No API changes. No new models.
- Reuse-first: every new surface composes existing stores, endpoints, and components.
- Redesign is re-presentation of existing data only — never new data plumbing.
- Pure computation is extracted as exported, unit-tested helpers.
- Each task is independently shippable and < 1 working day.
- This plan defines **how** implementation is executed. It generates no code itself.

Environment commands (verified against this repository):

- Typecheck: `npm run typecheck` (in `mainApp`)
- Frontend tests: `npm test` (in `mainApp`)
- Server tests: `npm test` (in `mainApp/server`) — must stay green on every merge
- Production build: `npm run build` (in `mainApp`)

How to use this document: every implementation task opens the Task Execution Template (§3), fills it from the ECIS task spec, and runs the gates in order (§2 workflow → §6 testing → §4/§5 review → §7 merge). Deviations from any rule here are governed by §10 — never silent.

1. Engineering Execution Rules

Hard rules. A task that violates a rule is not merged.

1.1 Structural rules (locked boundaries)

Rule	Statement
E1	No backend modification. <code>mainApp/server/**</code> (routes, models, middleware, migrations, jobs, index) is read-only unless explicitly approved via §10.
E2	No API addition or change. <code>src/utils/api.ts</code> and the server request/response contracts are frozen. New UI reads existing endpoints only.
E3	No database change. No new model, no schema change, no migration.
E4	No new route path contract change. Routes may be re-pointed per IA §8.7 (page retirements) using existing route components; URL shapes already in the sitemap are unchanged.
E5	No new architecture. Client store patterns (Zustand + optimistic <code>runMutation</code> + rollback), <code>loadCollabData</code> composition, and the pure-helper pattern are the only sanctioned patterns.

1.2 Reuse rules (the default)

Rule	Statement
R1	Reuse before write. Before any new component, search the codebase (UI kit, collaboration, worklog, layout, pages) and reuse. A new file requires justification in the task template.
R2	No duplicate components. One component per responsibility, one source of truth per surface. If a variant exists, adapt via props/wrapper (classification: R reuse-as-is, A reuse-with-adaptation, N new-composes-existing) — never copy-and-adapt into a sibling file.
R3	No duplicate selectors. All derivation lives in exported pure helpers in the owning store or a <code>lib/</code> module, unit-tested, imported by every consumer.
R4	No duplicated business logic. A behavior implemented once. Shared logic is a pure helper or store action; pages never re-implement each other's math.
R5	No duplicate navigation, palette mounts, or KPIs. One palette mount (<code>workspaceLayout</code>), one command palette, one computed source per KPI (existing <code>compute*</code> helpers). Clutter gate (ECIS H-8) is mandatory.

1.3 Quality rules

Rule	Statement
Q1	One responsibility per component and per page. Each surface answers exactly its primary question (IA §4). No page mixes a second surface's job.
Q2	Prefer composition. Sections are composed of existing primitives; large pages are section components fed by selectors.
Q3	No placeholder data. No fake metrics. If a value is not derivable from live store/API state, render  . Never fabricate numbers, mock members, or seed demo state in production code.
Q4	No hidden technical debt. A shortcut is either fixed in the same task or logged in §12 Decision Log with an owner and a follow-up task — never merged silently.
Q5	Honest states. Every data section specifies loading / empty / error / success per ECIS A.1. No unhandled states.
Q6	Calm & quiet by default. No blinking metrics, no interruptive admin on work surfaces, no notifications during Focus. Single primary action per screen.
Q7	Flow is sacred. Timer continuity is never broken by a UI change. Sessions pause, never abandon. Resume never double-starts a session.

1.4 Verification rules

Rule	Statement
V1	Every merge passes: typecheck, full frontend suite, full server suite (unchanged green), and any task-specific tests.
V2	Every pure helper has a unit test. Every new/modified surface has component tests covering loading/empty/error/success. Resume and session flows have integration tests.
V3	a11y assertions (axe) run on every new or modified surface.
V4	Server suite green = the server code was not changed. If a task <i>needs</i> a server change, it stops and escalates (§10.5) — it does not patch the server.

2. Development Workflow

The workflow every implementation task MUST follow, in order. Skipping a step is a defect.

1. Understand the requirement

- Read the ECIS screen/task spec (A.1–A.3 fields) + IA page responsibility + DCX behavior.
- Restate Objective and Primary Question in the task template. If ambiguous, ask before coding.

2. Review existing implementation

- Open the page(s)/component(s) the spec names (Today→Dashboard, Current Task→TaskDetail, Focus→FocusMode, Work Log→WorkLog/WorkLogDetail, Mission Control→TeamWorkspace, etc.).
- Read the owning stores (useStore / useAuthStore / useCollaborationStore) and the APIs in use.
- Read existing tests for the area. Match their patterns.

3. Locate reusable assets (in this order)

- a. Components → src/components/ui, src/components/{tasks,collaboration,worklog,layout}
- b. Selectors → exported pure helpers (compute*, select*, lib/*) and store getters
- c. Stores → useStore, useAuthStore, useCollaborationStore
- d. APIs → src/utils/api.ts (existing methods only)
- e. libs → dataMapper, docEngine, collaborationActivity, markdown, workspaceMaturity, config

4. Determine minimal changes

- Prefer: adapt an existing component (A) over a new one (N). Prefer a pure helper + section over a rewrite. A task that must touch 3+ pages without a strong reason is over-scoped.

5. Implement

- Small, shippable increments. New UI composes the reused assets. Pure logic in exported helpers. No server changes (E1–E3). Follow existing naming/conventions (ECIS A.5).

6. Test

- Unit tests for helpers; component tests for sections (loading/empty/error/success); integration tests for resume/session flows; all assertions. Run the local gates (§6.2).

7. Review

- Self-review against §5 Code Review Checklist. Update the task template with outcomes.
- Request review per §10 roles. Address all blockers.

8. Refactor

- Apply review feedback; re-run tests. No known debt left silent (§12 if unavoidable).

9. Merge

- PR per §4, then merge per §7 after the Release Manager gate.

Parallel note: steps 3–4 are the anti-duplication gate. Evidence of "searched and reused" must be recorded in the task template's Reuse Opportunities field, or the review is blocked.

3. Task Execution Template

Every implementation task opens this template, filled from the ECIS task spec (ECIS A.3 + the relevant B-screen). Fields marked **R** are required.

Field	Content
Task ID (R)	ECIS ID (e.g. S1-T2) — or EEP-ADD-<n> for work added under governance.
Objective (R)	One paragraph: what continuity behavior this delivers, from the ECIS/DCX text.
Primary Question (R)	The IA §4 question this surface answers.
Scope (R)	In: the screen/sections touched. Out: everything else (explicitly list non-goals).
Files expected to change (R)	Concrete paths. New file? Justified under Reuse Opportunities.
Files that must not change (R)	At minimum: <code>mainApp/server/**</code> , <code>src/utils/api.ts</code> , locked types/models. Add task-specific protected files (e.g. <code>timerEngine</code> if untouched).
Dependencies (R)	ECIS F/G dependencies, e.g. S1-T2 → S1-T1 . Task blocked until deps merge.
Reuse opportunities (R)	Evidence: which existing components/selectors/stores/APIs/libs were located and reused. "None" must be justified.
Implementation steps (R)	Ordered, small steps mapping to §2 workflow.
Acceptance criteria (R)	From ECIS §G + §H. Verifiable, no vague wording.
Testing requirements (R)	Which tests are mandatory (per §6): unit / integration / component / a11y / regression.
Regression checklist (R)	Routes still resolve; store actions unchanged; no duplicate mounts; timer rehydration correct; server suite green.
Definition of Done (R)	All ECIS §J gates pass; selectors pure + tested; no new API/model; reuse verified; server suite green; PR merged.
Review sign-off	Architecture · Frontend · QA · Release (names/date) per §10.

4. Pull Request Checklist

Merge-ready gate. Every PR must satisfy all items before the Release Manager merges.

- ☐ Single task per PR (one ECIS ID). No mixed concerns.
 - ☐ Title and body use the Git convention (§7).
 - ☐ **Files that must not change** (§3) are untouched — `git diff` verified against the protected list.
 - ☐ No server files in the diff (`mainApp/server/**`), unless an approved §10 exception is referenced.
 - ☐ Diff size is proportionate to the task (< 1 day of work; large diffs split).
 - ☐ Reuse evidence present in the task template (components/selectors/stores/APIs/libs reused).
 - ☐ No duplicated components/selectors/KPIs/nav introduced (R2–R5).
 - ☐ New pure logic is an exported, unit-tested helper (R3, V2).
 - ☐ No placeholder data, no fake metrics, honest `–` states (Q3).
 - ☐ Loading / empty / error / success states present per spec (Q5).
 - ☐ Accessibility: landmarks, heading order, labeled inputs, keyboard operable, single primary action, reduced-motion respected (ECIS §J).
 - ☐ Responsive: single column on mobile, nav drawer, tap targets $\geq 40\text{px}$, Now strip collapses (ECIS §J).
 - ☐ Performance: no re-fetch on tab switch, no layout thrash, motion $\leq 250\text{ ms}$ (ECIS §J).
 - ☐ Tests added/updated and passing locally: unit + component (+ integration/a11y where mandated).
 - ☐ Full frontend suite green: `npm test` (mainApp).
 - ☐ Full server suite green: `npm test` (mainApp/server).
 - ☐ Typecheck clean: `npm run typecheck` (mainApp).
 - ☐ `npm run build` (mainApp) succeeds.
 - ☐ No dead code, commented-out blocks, console noise, or secrets.
 - ☐ Manual smoke (documented in PR description): affected surface loads, resume/timer flows verified.
-

5. Code Review Checklist

Mandatory review dimensions. Reviewers comment on each dimension; "n/a" is allowed with a reason, silence is not.

#	Dimension	Check
1	Architecture consistency	Composes existing stores/endpoints; follows the pure-helper + optimistic-mutation patterns; no new architecture (E5).

#	Dimension	Check
2	Naming consistency	Matches codebase conventions (ECIS A.5); helper names are verbs/derived from data; routes match IA sitemap.
3	Component reuse	No reinvented primitive; no sibling duplicate of an existing component; composition over copy.
4	Selector purity	Derivation is pure, exported, unit-tested; no side effects in selectors; consumers don't re-derive.
5	Accessibility	Landmarks; heading order (h1 → h2 → ...); labeled inputs/selects; single primary action; focus management; contrast ≥ AA; live regions for timer/status; reduced motion.
6	Responsive behavior	Mobile single column; tables/boards scroll or stack; Now strip collapses; tap targets ≥ 40 px.
7	Performance	No blocking whole-page load where skeletons suffice; no re-fetch on tab switch (store cache); no layout thrash; motion GPU-friendly, ≤ 250 ms.
8	Error handling	Per-section inline retry; global banner via ErrorBoundary; no swallowed errors; honest failure surface.
9	Loading states	Skeleton per section, never a blank page; block only sections awaiting data.
10	Empty states	Meaningful <code>EmptyState</code> text per spec; – for non-derivable values.
11	Edge cases	No active task / no sprint / no members / no docs / paused-vs-stopped session / reload mid-session / task reassigned / multiple continuations ranked.
12	Tests	Helpers unit-tested; sections have loading/empty/error/success cases; resume/session flows integration-tested; a11y assertions present.
13	Regression	Existing routes resolve; collab + personal suites green; no duplicate palette/timer mounts; server suite green.
14	Documentation	Task template updated with outcomes; decisions logged in §12; README/integration notes touched only when truly required.

Review grading: **Block** (must fix before merge) · **Suggest** (fix within the PR if cheap, else §12 follow-up) · **Approve**.

6. Testing Strategy

6.1 Test levels and mandatory status

Level	What it covers	Mandatory?
Unit	Every exported pure helper (<code>select*</code> , <code>compute*</code> , <code>deriveNow</code> , <code>selectMemory</code> , <code>selectKnowledge</code> , <code>selectTaskContinuation</code> , <code>selectSessionState</code> , <code>resume resolver</code>). Pure input → pure output, edge cases (– states, empty arrays).	Always
Integration	Resume flow (one tap restores task + session + work log + branch/PR), session rehydration from <code>timerPersist</code> + <code>sessions.list({active:true})</code> , idempotency via <code>clientOpId</code> (no double-start).	Always for S1-T4/S1-T5 and any session/timer-touching task
Component (UI)	Each new/modified section renders loading / empty / error / success; store-driven data appears; interactions (checkboxes, selects, buttons, palette) work.	Always for new surfaces
Accessibility	axe assertions on new/modified surfaces; heading order; labels; single primary action.	Always for new/modified surfaces
Regression	Existing routes resolve; store actions unchanged; no duplicate mounts (palette, timers); timer rehydration correct; personal + collab suites green; server suite green.	Always
Manual verification	Documented smoke per PR: affected surfaces load, resume/timer flows work, mobile layout sane, empty/error states visible.	Always (release gate)
Performance	Measured only where a task touches hot paths (Today, lists, Mission Control): no re-fetch on tab switch, no layout thrash, lazy routes verified.	When flagged in task risk (H/M perf)

6.2 Local gate order (run in every task)

1. `npm run typecheck` (mainApp) — clean.
2. Targeted tests for the change (unit + component + integration).
3. `npm test` (mainApp) — full frontend suite green.
4. `npm test` (mainApp/server) — full server suite green, unchanged.
5. `npm run build` (mainApp) — production build succeeds.

6.3 Testing conventions

- Pure helpers live next to their source (store file or `lib/`) as exported functions and are tested in dedicated suites matching existing test layout (`src/**/*.test.ts`).
 - Component tests follow existing patterns (vitest + happy-dom), mocking only `src/Utils/api` and stores — never fabricating backend behavior beyond the mock.
 - Tests assert honest — behavior: empty/underviable data renders the placeholder, not zero.
-

7. Git Workflow

7.1 Per-task sequence

- | | |
|----------------------------|--|
| 1. Task selection | → From the ordered backlog (§8). One task, one branch. |
| 2. Branch | → <code>feature/<ecis-id>-<kebab-slug></code> , e.g. <code>feature/s1-t2-today-page</code> . |
| 3. Implementation | → Per §2 workflow and §3 template. |
| 4. Local testing | → §6.2 gate order (typecheck → targeted → frontend → server → build). |
| 5. Manual verification | → Smoke the affected surface per §6.1. |
| 6. Commit | → §7.2 convention. Multiple logical commits allowed per task, each green. |
| 7. Push | → Push branch; open PR against the integration branch. |
| 8. Review | → §4 + §5. Resolve all Blocks. |
| 9. Merge (linear history). | → Release Manager merges after all gates green. Rebase-merge |
| 10. Follow-up | → Update §12 Decision Log if any debt/exception was logged. |

7.2 Commit message conventions

Format: `<type>(<scope>): <summary>` (imperative, ≤ 72 chars), body free-form with rationale and optional `Refs: <ECIS task id>`.

- Types: `feat`, `fix`, `refactor`, `test`, `docs`, `chore`, `perf`, `revert`.
- Scopes: `ui`, `collab`, `worklog`, `timer`, `nav`, `reports`, `a11y`, `store`, `today`, `focus`, `mission-control`, `knowledge`.
- Examples (matching repo history):
 - `feat(today): Today landing with Continue / Do now / Attention (S1-T2)`
 - `refactor(nav): split TeamWorkspace mega-tabs into routed pages (S4-T1)`
 - `test(store): selectToday/selectDoNow/selectAttention pure helpers (S1-T1)`

Rules:

- One concern per commit; commits in a PR may be squashed at merge to one per task.
 - No WIP/no-op commits, no binary/secrets, no large generated files.
 - Never amend a pushed commit. Never force-push. Never commit directly to the integration branch.
-

8. Implementation Order & Parallelization Strategy

Confirmed sequence = ECIS Sprint Plan (§G) and IA §8.10, unchanged. Each task is an independent, shippable increment; server suite stays green throughout.

8.1 Sequence and dependencies

S1 Continuity core (P0) – the foundation; lands first, sequentially.
S1-T1 selectToday / selectContinue / selectDoNow / selectAttention [anchor, no deps]
S1-T2 TodayPage sections (Header/Continue/Focus/Attention) [← S1-T1]
S1-T3 NowStrip (task · clock · subtasks · branch/PR) [← S1-T1]
S1-T4 selectSessionState + rehydration + no-double-start [← S1-T3]
S1-T5 Resume flow (one tap → Current Task + session + work log) [← S1-T3, S1-T4]

S2 Working loop (P0/P1)
S2-T1 Current Task continuation view (Now strip, subtasks, session, git, where I stopped) [← S1-T5]
S2-T2 Focus shell: inline blocker + decision capture on intentional pause [← S2-T1]
S2-T3 Focus completion prompt → reflection (journal + completed item) [← S2-T2]

S3 Memory & knowledge (P0/P1/P2)
S3-T1 Work Log master/detail merge, "Where I stopped" highlighted [no deps – see 8.3]
S3-T2 selectKnowledge + Knowledge surface [← S3-T1]
S3-T3 Personal Reports merge (analytics → views) [no deps]
S3-T4 Mission Control tune (lead with Today/resume/running timer) [← S1-T1; serialized with S4-T1 – see 8.3]

S4 Team split + cleanup (P1/P2)
S4-T1 Split TeamWorkspace mega-tabs → routed Sprint/Backlog/Blockers pages [no deps – serialized with S3-T4]
S4-T2 Merge ReportsAnalyticsPage analytics view; canonicalize KPIs [no deps]
S4-T3 L3 administration extraction (Projects/Teams/Members/Settings/Audit) [no deps]
S4-T4 Role-aware default navigation + /team route collision cleanup [no deps]

8.2 Parallel tracks (safe concurrency)

Two developers may work in parallel on **disjoint files**:

Track A	Track B	Non-overlap guard
S1-T1 (helpers only)	S3-T1 (Work Log merge)	helpers/tests vs WorkLog page/lib
S1-T2 (TodayPage)	S2-T1 preparation on TaskDetail after S1-T5 merges	different pages
S1-T3 (NowStrip)	S3-T3 (Reports merge)	AppLayout/NowStrip vs Reports

Track A	Track B	Non-overlap guard
S3-T1 (Work Log)	S3-T3 (Reports)	WorkLog vs Reports
S2-T2 / S2-T3 (Focus)	S4-T3 (L3 extraction)	FocusMode vs admin/layout pages
S3-T2 (Knowledge)	S4-T2 (KPI canonicalization)	Knowledge surface vs ReportsAnalyticsPage

Concurrency rule: two tasks may run in parallel only if their **Files expected to change** sets are disjoint. Anything touching `AppLayout` , `WorkspaceLayout` , `TeamWorkspace` , `FocusMode` , or `src/utils/api.ts` serializes with everything else that touches the same file.

8.3 Blockers (hard sequencing)

- **S1-T1 blocks S1-T2 and S1-T3.** It is the anchor and must merge first.
- **S1-T4/T1-T5 block S2.** Current Task and Resume consume session rehydration.
- **S3-T4 and S4-T1 are mutually exclusive on `TeamWorkspace`.** ECIS orders S3-T4 first; the mega-tab split (S4-T1) then extracts the remaining tabs into routed pages. These two tasks cannot be parallelized.
- No task is blocked on a new API, model, or backend change — by construction (locked).

8.4 Tasks requiring architectural review (gate before merge, §10.4)

Task	Why it needs review
S1-T4 (session rehydration / no double-start)	Touches timer correctness and idempotency — the core continuity guarantee (Q7).
S1-T5 (Resume flow)	Introduces a resume intent resolver and multi-surface navigation contract.
S3-T1 (Work Log master/detail merge)	Page architecture change (IA §8.7-4).
S4-T1 (mega-tab split)	Routing/navigation architecture; highest regression surface (all L2 pages).
S4-T3 (L3 extraction)	Navigation/role boundary change.
S4-T4 (role-aware default nav)	Navigation default-behavior change across roles.
Any task touching <code>AppLayout</code> , <code>WorkspaceLayout</code> , or <code>FocusMode</code>	Shell-level change.

9. Risk Register

ID	Risk	Category	L	I	Mitigation
R K-1	Component duplication (developer copies an existing component instead of adapting it)	Reuse	H	M	§2 steps 3–4 are mandatory; reuse evidence in template; §5 review dimension 3 blocks duplicate commits.
R K-2	Selector/business-logic duplication (derivation re-implemented in pages)	Reuse	H	M	Pure-helper rule R3/R4; unit tests on helpers; review dimension 4.
R K-3	State inconsistency between personal (<code>useStore</code>) and collab (<code>useCollaborationStore</code>) views of the same task/session	State	M	H	Single shared spine (task/session/worklog) per DCX §2; NowStrip composes both stores from one derivation; integration tests on resume/session.
R K-4	Session double-start / lost time on reload or context switch	Continuity	M	H	S1-T4 idempotency via <code>clientOpId</code> ; rehydration from <code>timerPersist</code> + <code>sessions.list({active:true})</code> ; integration test; review gate on S1-T4.
R K-5	Navigation regressions from the mega-tab split / route retirement (IA §8.7)	Navigation	H	H	S4-T1 serialized, review-gated; full regression checklist per PR; route smoke of every retired route.
R K-6	Performance regression on Today / lists / Mission Control (re-fetch, layout thrash)	Performance	M	M	Skeleton over blocking; store-cached data (no re-fetch on tab switch); motion ≤ 250 ms; perf check where flagged.
R K-7	Context loss across context switches (project/sprint re-frame resets thread)	Continuity	M	H	Re-frame, never reset (IA §5); NowStrip survives transitions; breadcrumb spine; tests for task/session continuity across nav.
R K-8	Accessibility regression (unlabeled controls, focus loss, contrast, reduced-motion)	A11y	M	M	§5 dimension 5; axe assertions mandatory; single primary action rule.
R K-9	Clutter regression (duplicate palette mounts, duplicate KPIs, global buttons reappear)	Clutter	M	L	R5 + ECIS H-8 clutter gate; regression checklist item "no duplicate mounts".

ID	Risk	Category	L	I	Mitigation
R K- 10	Boundary creep (a task silently touches server/API/model to "make it work")	Locked scope	M	H	E1–E3 + §10.5 escalation; "Files that must not change" verified in every PR; server suite green as tripwire.
R K- 11	Empty/loading/error states missing on new sections	UX integrity	M	M	Q5 + §5 dimensions 8–10; component tests for all four states.
R K- 12	Honest-data violations (fabricated metrics or placeholder seed data in production)	Data integrity	M	H	Q3; – rule; unit tests asserting placeholder behavior.
R K- 13	Merge conflicts on shared shell files (AppLayout , WorkspaceLayout , TeamWorkspace , FocusMode)	Delivery	M	M	§8.2 concurrency guard: disjoint-file rule; serialization on shell files.
R K- 14	Context switching a11y of NowStrip across surfaces (focus management)	A11y	L	M	Focus moves to section on load; keyboard operable; live regions for clock state.

Risk scoring: L = Likelihood (H/M/L), I = Impact (H/M/L). Any task flagged H/H or M/H in its template must receive architectural review (§10.4) before merge.

10. Implementation Governance

10.1 Roles and gates

Role	Responsibility	Gate
Technical Program Manager (TPM)	Maintains §8 order and the backlog; approves task selection and deps; owns §12 Decision Log.	Task start; sequence changes.
Staff Software Engineer (Architecture)	Reviews architecture-flagged tasks (§8.4); approves patterns; enforces E1–E5.	Architecture review gate.
Principal Frontend Architect	Reviews component reuse, composition, IA fidelity, NowStrip/shell changes.	Frontend review gate.

Role	Responsibility	Gate
QA Lead	Enforces §6 testing strategy; verifies gates and regression checklist; runs release verification.	Test gate.
Release Manager	Final PR gate (§4); verifies server suite green and protected paths untouched; merges.	Merge gate.

10.2 Definition of approved

"Approved" for any locked-boundary exception means: documented in §12 with an explicit decision by the Architecture + Release roles, referencing the ECIS/IA constraint it overrides. Absence of an entry = not approved.

10.3 Change control

- Any change to the four locked documents or to this EEP requires a §12 entry by the TPM before it takes effect. No silent edits.
- Task ordering may only be adjusted via §12 entries that respect §8.3 blockers.

10.4 Architectural review gate

Tasks in §8.4 (or flagged H in the risk register) do not merge until Architecture review approves. Review outcome recorded in the task template.

10.5 Escalation (boundary creep)

If a task discovers it *needs* a backend/API/model change to satisfy its objective:

- Stop implementation.
- Log a §12 entry: what the gap is, the ECIS task, and the data needed.
- Do **not** patch the server. Instead, degrade gracefully (render honest `–` /empty state) so the task remains shippable.
- The governance board decides whether a locked exception is warranted. No exception, no change.

10.6 Release discipline

- MVP = ECIS S1–S4 complete with the server suite green throughout (ECIS §I).
 - Each sprint ends with a green integration branch, §6.2 gates passing, and §12 current.
 - No release ships with known Silent Debt (§12 entries without a follow-up task).
-

11. Appendix A — Reuse Inventory (index, not a substitute for searching)

Verified present in this repository. Search before writing anything new.

UI kit (`src/components/ui`): `Card`, `Button`, `Badge`, `Input`, `Select`, `Textarea`, `Spinner`, `Skeleton`, `Progress`, `Tooltip`, `StatusBadge`, `Breadcrumbs`, `PageHeader`, `EmptyState`, `StandardEmptyState`, `ToastContainer`, `Dialog`, `Field`, `ErrorBoundary`, `GlobalHeader`, `WorkspaceBadge`, `FocusFlowLogo`, `ThemeToggle`.

Layout (`src/components/layout`): `AppLayout`, `WorkspaceLayout` (grouped sidebar + identity block + Mission Control label), plus admin layout/sidebar.

Personal companion machinery (`src`, `src/components/worklog`, `src/utils`): timer engine (`activeTimerState`, `currentSessionStart`, `currentPauseStart`, `timerPersist`), session client + server reaper, `WorkLog` full schema and sub-views (`WorkLogWidget`, `TimelineView`, `TechnicalDecisionsView`, `StructuredBlockersView`, `ReflectionView`, `TomorrowPlanView`, `ProblemFlowEditor`, `AttachmentsView`, `ReadingModeView`, `WorkLogExporterModal`), report routes (`summary` / `day` / `share` / `leaderboard`), pages `Dashboard`, `FocusMode`, `TaskDetail`, `WorkLog`, `WorkLogDetail`, `Journal`, `Habits`, `Reports`, `Analytics`, `SearchResults`, `Settings`, `Landing`, `Login`, `Register`.

Collaboration machinery (`src/pages/collaboration`, `src/components/collaboration`): `TeamWorkspace` (Mission Control default tab) + pure helpers (`computeSprintVelocity`, `computeWorkspaceProgress`, `computePendingReviews`, `computeAssignedWork`, `computeUpcomingDeadlines`); `WorkspaceLayout` grouped nav; `ProjectBacklog`, `WorkItemTypeBadge`, `FeaturesPage`; create modals (`CreateProjectModal`, `CreateSprintModal`, `CreateTaskModal`, `CreateFeatureModal`, `CreateBlockerModal`, `CreateDocModal`); `ReportsAnalyticsPage`, `ActivityFeedPage`, `QADashboardPage`, `MemberProfilePage`, `WorkspaceSettingsPage`, `DiscussionsModal`, `NotificationCenter`, `GlobalCommandPalette`.

Stores & libs: `useStore`, `useAuthStore`, `useCollaborationStore` (Zustand + optimistic `runMutation` / `rollback`, `loadCollabData` composition); `lib/collaborationActivity` (`activityActionLabel` / `activityDetail`), `lib/dataMapper`, `lib/docEngine`, `lib/markdown`, `utils/workspaceMaturity`, `utils/config`.

New components sanctioned by ECIS §C (compose existing — small, testable): `TodayPage`, `NowStrip`, `ResumeFlow` (pure resolver), and pure helpers `selectToday`, `selectContinue`, `selectDoNow`, `selectAttention`, `selectTaskContinuation`, `selectMemory`, `selectKnowledge`, `selectSessionState`.

12. Appendix B — Decision Log (append-only)

Deviations, exceptions, approved scope changes, and silent-debt entries are recorded here. An entry must include: date, task ID, decision, rationale, approving roles, and (for debt) a follow-up task ID. This log is the single place governance state lives.

#	Da te	Ta sk	Decision	Rationale	Approvers	Follo w-up
(st art)	—	—	EEP ratified as final engineering document.	Locked docs require an operating manual; no further planning.	TPM · Arch · FE · QA · Release	—

End of Engineering Execution Plan. The next phase is implementation only.